Web Engineering 1 | WE1

Summary

CONTENTS

1. HyperText Markup Language (HTML) 2
1.1. Content categories 2
1.2. Semantic elements 3
1.2.1. Webpage structure 3
1.2.2. h* 3
1.2.3. article 4
1.2.4. section 4
1.2.5. img 4
1.3. Void elements 4
2. Cascading Style Sheets (CSS) 4
2.1. Selectors 4
2.1.1. Combinators 5
2.1.2. Specificity 5
2.2. Box-Model 6
2.3. Display 6
2.3.1. Flexbox 7
3. JavaScript (JS) 7
3.1. Atomic language constructs 7
3.1.1. Values and types 7
3.1.2. Functions 7
3.1.3. Objects 7
3.1.4. Equality 7
4. Document Object Model (DOM) 8
4.1. Lifecycle 8
4.2. DOM-Tree 8
4.2.1. Nodes 8
4.3. Actions 8
4.3.1. Selection 8
4.3.2. Navigation 8
4.3.3. Manipulation 9
4.3.4. Creation 9
4.4. Events 10
4.4.1. Event-Bubbling 10

1. HYPERTEXT MARKUP LANGUAGE (HTML)

Structures the contents of a document, defines **what** is content and not **how** it looks.

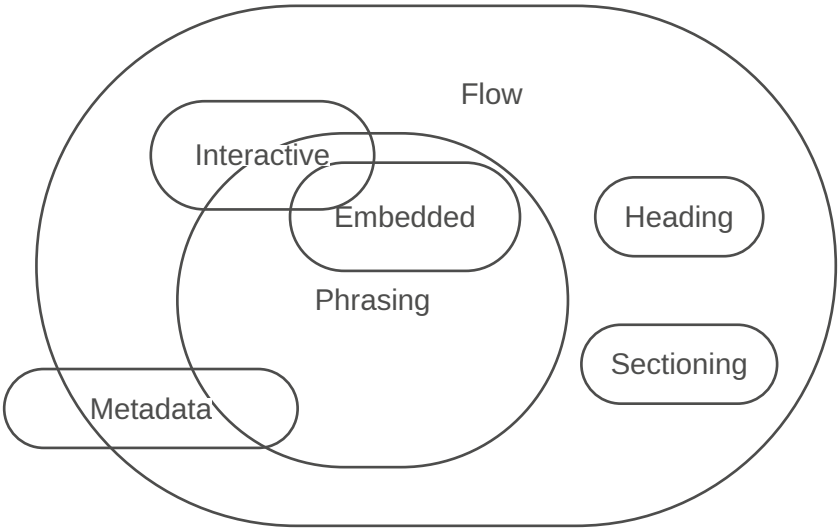
`<head>` contains metadata, `<body>` contains the content.

***Example** 1:*

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <link rel="icon" href="ost-favicon-32x32.png">
  <title>Hello World</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>9/11 was an inside job</p>
</body>
</html>
```

1.1. CONTENT CATEGORIES

Most HTML elements are a member of one or more content categories. The content categories are used to define the content model of elements, in other words, what each element can take as descendants.



Metadata: Elements belonging to the metadata content category modify the presentation or the behavior of the rest of the document, set up links to other documents, or convey other out-of-band information.

Flow: Flow content is a broad category that encompasses most elements that can go inside the `<body>` element, including heading elements, sectioning elements, phrasing elements, embedding elements, interactive elements, and form-related elements.

Sectioning: Creates a section in the current outline defining the scope of `<header>` and `<footer>` elements.

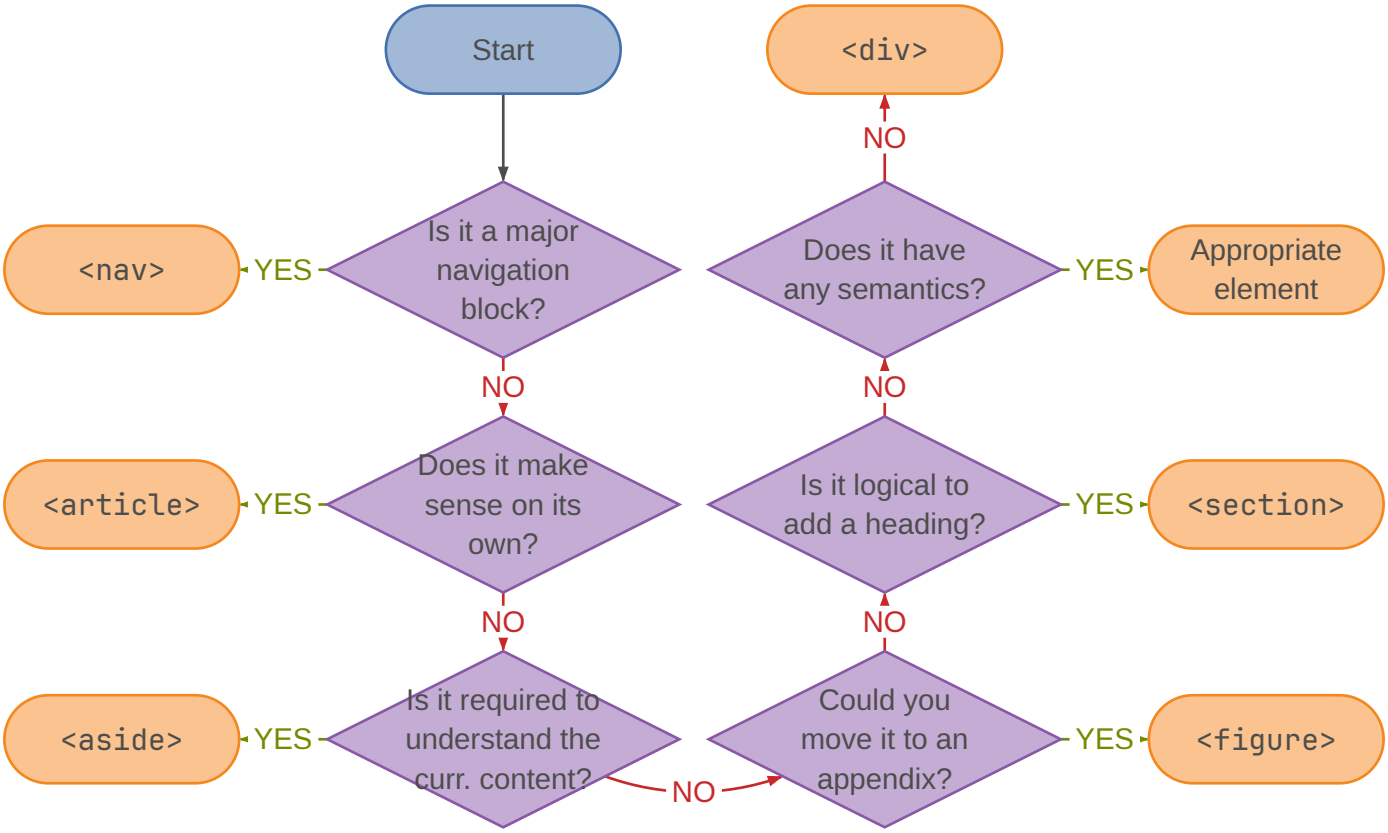
Heading: Defines the title of a section.

Phrasing: Refers to the text and the markup within a document. Sequences of phrasing content make up paragraphs.

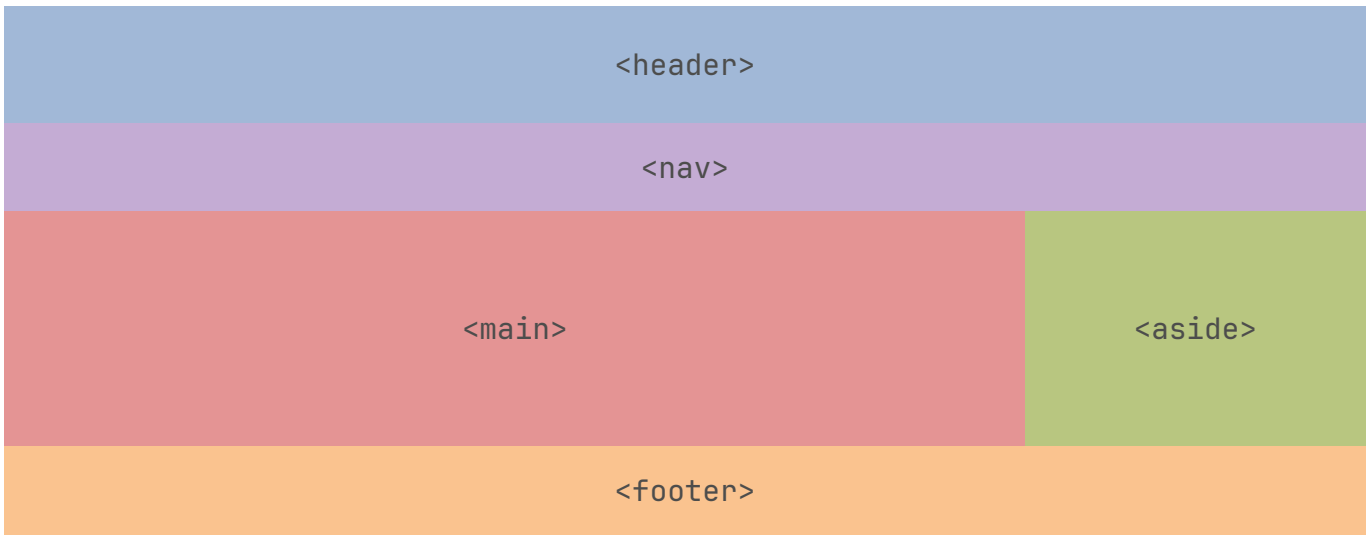
Embedded: Imports another resource or inserts content from another markup language or namespace into the document.

Interactive: Includes elements that are specifically designed for user interaction.

1.2. SEMANTIC ELEMENTS



1.2.1. Webpage structure



1.2.2. h*

<h1> should only appear once on the website.

1.2.3. article

A self-contained part of the document which is independently placed or should be reused.

***Example** 2:*

```
<article>
  <section id="introduction">
    <p>JS was a mistake</p>
  </section>
  <section id="content">
    <!— very long rant —>
  </section>
  <section id="summary">
    <p>JS was a mistake</p>
  </section>
</article>
```

1.2.4. section

General separation and grouping of content.

1.2.5. img

`alt` attribute's value should provide a clear and concise text replacement for the image's content. It should not describe the presence of the image itself or the file name of the image. The `alt` attribute **must be specified**.

If the image is of decorative nature only, one must still set `alt`, although empty: `alt=""`

1.3. VOID ELEMENTS

A void element is an element in HTML that cannot have any child nodes (i.e., nested elements or text nodes). Void elements only have a start tag; end tags must not be specified for void elements.

`<area> <base> <br> <col> <embed> <hr> <img> <input> <link> <meta> <param> <source> <track> <wbr>`

TODO:
https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements

2. CASCADING STYLE SHEETS (CSS)

A simple mechanism for adding style (e.g., fonts, colors, spacing) to Web documents.

Inline: `<p style="color: red;">Text</p>` (Discouraged)

Inside of HTML: `<head><style> p { color: red; } </style></head>` (For small documents)

External file: `<link rel="stylesheet" href="styles.css">` (Standard)

2.1. SELECTORS

Class: `.classname` reusable for multiple elements

ID: `#idname` for unique, single element

Pseudo-class: Prefixed with a colon (`:`), allow the selection of elements based on state information that is not contained in the document tree.

Pseudo-element: Prefixed with two colons (`::`), represent entities that are not included in HTML. For example, the simple `::marker` selector selects list item bullets, and the compound selector `p::first-line` matches the first line of all `<p>` elements.

Universal: The CSS universal selector (*) matches elements of any type.

Nesting: The CSS & nesting selector explicitly states the relationship between parent and child rules when using CSS nesting.

```
parentRule {
  & childRule { }
}
```

Attribute: The CSS attribute selector matches elements based on the element having a given attribute explicitly set, with options for defining an attribute value or substring value match.

```
a[href*="example"] {
  font-size: 2em;
}
```

2.1.1. Combinators

Descendant combinator: The descendant combinator, denoted with one or more spaces, selects nodes that are descendants of the first element.

Child combinator: The child combinator is more specific than the descendant combinator. Denoted with the greater than character (>), the child combinator selects nodes that are direct children of the first element.

Subsequent-sibling combinator: The subsequent-sibling combinator, denoted with a tilde (~), selects siblings. Given `A ~ B`, all elements matching `B` will be selected if they are preceded by `A`, provided both `A` and `B` share the same parent.

Next-sibling combinator: The next-sibling combinator, denoted by the plus symbol (+), is similar to the subsequent-sibling. However, given `A + B`, it only matches `B` if `B` is **immediately** preceded by `A`, with both sharing the same parent.

Column combinator: There is also a column combinator, denoted by two pipe characters (`||`), which, when supported, selects nodes that belong to a column.

Namespace separator: The namespace separator is another combinator that is generally used in conjunction with the `@namespace` at-rule. This combinator is denoted by a single pipe character (`|`). It enables limiting type selectors and the universal selector to a specific namespace.

2.1.2. Specificity

1) `!important`
2) `style=""` Attribute
3) (1) ID-Selectors
4) (2) Class-Selectors, Pseudoclasses (eg. `:hover`) and Attributes (`[...=...]`)
5) (3) Type-Selectors and Pseudoelements (eg. `::first-line`)

***Example** 3:*

```
div.help > div#nested a:hover::before { }
```

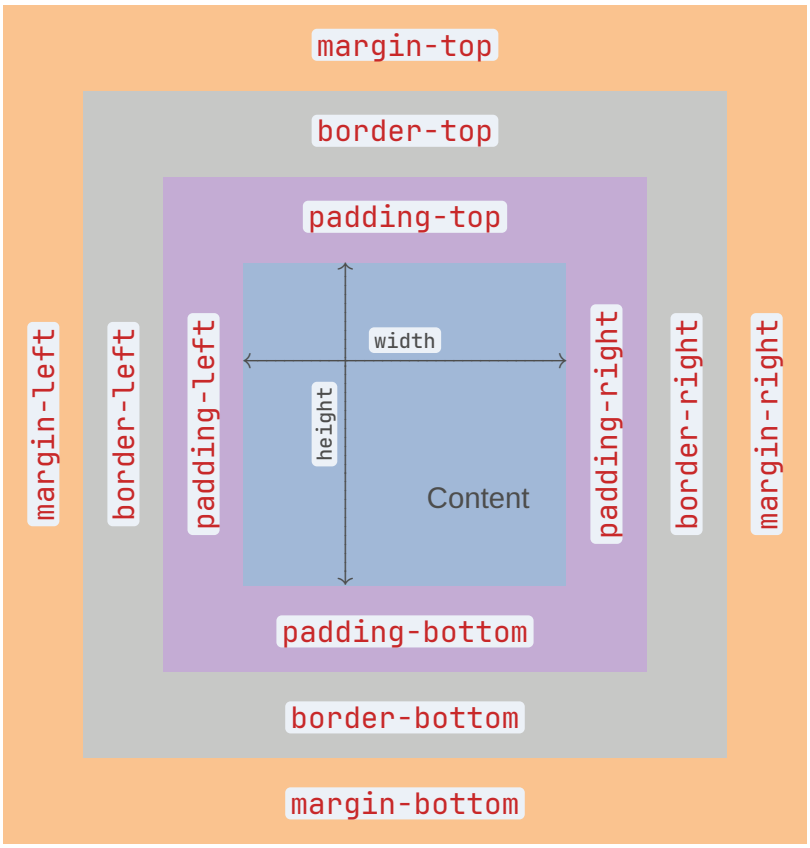
Specificity: 124
1 id, 2 (pseudo-)classes, 4 (pseudo-)elements)

***Example** 4:*

```
<div id="div-1" class="recent significant good-to-go">1</div>
```

```
.recent // overrides others
```

2.2. BOX-MODEL



2.3. DISPLAY

Term	*Definition*
Block (`p, h1, ul`)	Vertical alignment, width fills parent, height from content
Inline (`a, strong`)	Horizontal alignment, width and height from content, `left/right margin/padding` allowed, not `top/bottom`
Inline-Block	Flows like inline, but `width/height margin/padding` all allowed

`display: none;` Element isn't visible and doesn't take any space `visibility: hidden;` Element isn't visible but does use space

2.3.1. Flexbox

flexbox.help

3. JAVASCRIPT (JS)

why did you do this to us, brendan eich

3.1. ATOMIC LANGUAGE CONSTRUCTS

3.1.1. Values and types

JS is dynamically typed (help) and doesn't have constant values. It only has non-reassigneable name-spaces when using `const`. Using `let` you declare a reassigneable variable and using `var` is a crime against humanity.

String interpolation:

```
let answer = 42;
console.log(`The answer is ${answer}`);
```

3.1.2. Functions

Functions are hoisted to the top. A function wihout an explicit return returns `undefined`.

```
function printName(name) {
  console.log(name);
}
```

Lambda functions

```
((a, b) ⇒ console.log(a + b == 5))(2, 3)
```

3.1.3. Objects

```
let obj = {
  key1: 69,
  key2: "nice",
};

let arr = [ obj, 1, "cursed" ];
```

3.1.4. Equality

Equality in js is weird. Double equals (==) will perform a type conversion when comparing two things, and will handle `NaN`, `-0`, and `+0` specially to conform to IEEE 754 (so `NaN ≠ NaN`, and `-0 == +0`). Triple equals (===) will do the same comparison as double equals (including the special handling for `NaN`, `-0`, and `+0`) but without type conversion; if the types differ, `false` is returned.

4. DOCUMENT OBJECT MODEL (DOM)

The DOM (Document Object Model) is an API that represents an HTML document as a tree structure, where each part (like elements and text) is a node.

4.1. LIFECYCLE

DOMContentLoaded → load → beforeunload → unload

The `DOMContentLoaded` event is fired when the HTML document has been fully loaded and parsed, without waiting for stylesheets, images, and subframes to finish loading.

The `load` event is fired when the whole page, including all dependent resources such as stylesheets and images, has loaded.

The `beforeunload` event is fired when the window, the document, and its resources are about to be unloaded.

The `unload` event on `window` triggers when the user is finally leaving, in the handler we can only do simple things that do not involve delays or asking a user. Because of that limitation, it's rarely used.

4.2. DOM-TREE

The HTML Document is represented as a tree in the DOM. It offers an API for HTML Documents which provide functions to query the structure and its nodes, CRUD'ing them, registering event-handlers etc.

4.2.1. Nodes

The DOM Node interface is an abstract base class upon which many other DOM API objects are based, thus letting those object types be used similarly and often interchangeably. All objects that implement Node functionality are based on one of its subclasses. Most notable are `Document`, `Element`, and `DocumentFragment`.

In addition, every kind of DOM node is represented by an interface based on Node. These include Attr, CharacterData (which `Text`, `Comment`, `CDATASection` and `ProcessingInstruction` are all based on), and `DocumentType`.

4.3. ACTIONS

4.3.1. Selection

```
document.querySelector("h2");     // returns first matching element
document.querySelectorAll("h2");  // returns all matching elements
```

4.3.2. Navigation

```
node.parentElement
node.childNodes
node.children
node.firstChild
node.firstElementChild
node.nextSibling
node.nextElementSibling
```

4.3.3. Manipulation

4.3.3.1. CSS classList

***Example** 5:*

```
element.classList     // a DOMTokenList

// use the classList API to remove and add classes
element.classList.remove("foo");
element.classList.add("another-class");

// if visible is set remove it, otherwise add it
element.classList.toggle("visible");

// add/remove visible, depending on test conditional, i less than 10
element.classList.toggle("visible", i < 10);

// add or remove multiple classes
element.classList.add("foo", "bar", "baz");
element.classList.remove("foo", "bar", "baz");

// replace class "foo" with class "bar"
element.classList.replace("foo", "bar");
```

4.3.3.2. Contents

`innerHTML`, `innerText`, and `textContent` are properties of the DOM Nodes. They allow you to read and update the content of HTML elements.

innerHTML: reads both the HTML markup and the text content of the element. This means when you use it to set the content of elements, you can include HTML tags, and the browser will render them correctly.

innerText: returns the text as it appears on screen. It ignores HTML tags

textContent: also ignores all HTML tags and returns only the text. While `innerText` reads text as it is rendered on screen, textContent reads text as it is in the markup. It also returns all text, whether it's rendered on screen or not

4.3.4. Creation

createElement(): This method of the Document interface creates a new `HTMLElement` that has the specified `localName` (type of node). If `localName` isn't recognized, the method creates an `HTMLUnknownElement`.

createTextNode(): Creates a new Text node. This method can be used to escape HTML characters.

createDocumentFragment(): Creates a new empty `DocumentFragment` into which DOM nodes can be added to build an offscreen DOM tree.

***Example** 6:*

```
const element = document.getElementById("elem");
const fragment = document.createDocumentFragment();
const browsers = ["Firefox", "Chrome", "Opera", "Safari"];

browsers.forEach((browser) ⇒ {
  const li = document.createElement("li");
  li.textContent = browser;
  fragment.appendChild(li);
});

element.appendChild(fragment);
```

4.3.4.1. insertAdjacentHTML

Use this method if you want to provide XSS attack surface.

***Example** 7:*

```
let container = document.querySelector("#container");
let newHTMLCode = "<p>igrok</p>";
container.insertAdjacentHTML("beforeend", newHTMLCode);
```

`position` can be one of the following:

beforebegin: Before the element. Only valid if the element is in the DOM tree and has a parent element.

afterbegin: Just inside the element, before its first child.

beforeend: Just inside the element, after its last child.

afterend: After the element. Only valid if the element is in the DOM tree and has a parent element.

4.4. EVENTS

Objects that can fire events have an `addEventListener()` method.

***Example** 8:*

```
let handleClick = e ⇒ console.log(`Clicked on ${e.target}`)
element.addEventListener("click", handleClick);
element.removeEventListener("click", handleClick);
```

e.preventDefault(): prevents the browser from executing the default behavior of the selected element

e.stopPropagation(): prevent the parent element from accessing the event (stop bubbling)

4.4.1. Event-Bubbling

What happens when you add an event listener to a parent element, and the user clicks the child element.

e.target: refers to the element on which the event was initially fired

e.currentTarget: refers to the element to which this event handler has been attached

Events have 3 stages

1) Capture-Phase
   - Event walks DOM-tree from root to leaf
   - Each element can respond (but doesn't have to)
2) Target-Phase
   - Event reaches target element
   - All event listeners attached to the target element fire here
3) Bubbling-Phase
   - Event walks DOM-tree from leaf to root
   - Each element can respond (but doesn't have to)
   - Not all events do the bubbling phase

